BIRLA VISHVAKARMA MAHAVIDYALAYA (ENGINEERING COLLEGE)

(An Autonomous Institute)

Computer Engineering Department Academic Year: 2025-26

A PROJECT REPORT ON

(202CP - DBMS)

DATABASE MANAGEMENT SYSTEM

Submitted by:

Jeemit Patel
24CP027
Jeet Vasoya
24CP028

Guided by:

Prof. Kirtikumar J. Sharma

Birla Vishvakarma Mahavidyalaya (BVM), Vallabh Vidyanagar

NOVEMBER, 2025

CERTIFICATE

This is to certify that **Jeemit Patel** and **Jeet Vasoya**, bearing ID Numbers **24CP027** and **24CP028**, have successfully completed the course **202CP – Database Management System** and presented a seminar project titled **"Site Management System"** as part of the **Third Semester Curriculum** of the **Computer Engineering Program** at **Birla Vishvakarma Mahavidyalaya (BVM) Engineering College**.

The students have successfully **submitted and presented** their seminar work to the satisfaction of the evaluation committee.

This certificate is awarded in recognition of their **dedication, performance, and successful completion** of the seminar requirements.

Guide Name:

Prof . Kirtikumar J Sharma
Designation
Computer Engineering
Department BVM
Engineering College

Date: 11th November,2025
Place: BVM Engineering College, Vallabh Vidyanagar

ABSTRACT

The Site Management System is a database-driven application developed to streamline the operations involved in managing construction projects. It focuses on centralizing data related to users, projects, workers, materials, vendors, and finances, ensuring efficient tracking and coordination across all site activities. The system enables role-based access for administrators, project managers, site engineers, and storekeepers to handle tasks such as project creation, task assignment, worker attendance, material procurement, and financial record management.

By maintaining normalized relational tables and enforcing data integrity through foreign key constraints, the system ensures consistency, eliminates redundancy, and enhances reliability in data handling. The backend is implemented using ***PostgreSQL****, leveraging SQL queries, procedures, cursors, and triggers to automate updates and generate analytical reports like project expenses, vendor costs, and budget utilization.*

Overall, this system provides a comprehensive solution for construction project monitoring, improving operational transparency, reducing manual errors, and supporting data-driven decision-making for effective site management.

TABLE OF CONTENTS

CERTIFICATE II

ABSTRACT III

Chapter 1: Functional Requirements 1

Chapter 2: System Design 3

2.1 Entity Sets 3

2.2 Relationship Sets 4

2.3 Entity Relationship Diagram 6

2.4 Relational Table 7

2.5 Normalization 9

2.6 Relational Model 14

Chapter 3: Schema in PostgreSQL 15

3.1 Create Tables & Insert Records 15

3.2 Queries With Outputs To Fetch Vital Records From Database 27

CHAPTER 1

1. FUNCTIONAL REQUIREMENT

- Users will be able to create their own accounts and log in according to their roles such as Admin, Project Manager, Site Engineer, or Storekeeper.
- Admin will be responsible for adding new users, assigning their roles, and managing overall access of the system.
- Admin or project manager will be able to create a new project by entering project details like project name, location, type, start and end dates, and estimated budget.
- Each project can have multiple team members assigned such as engineers, storekeepers, and site supervisors depending on the requirement.
- Project managers will be able to assign different tasks under each project with start date, end date, task name, and status.
- Site engineers will be able to update the progress of assigned tasks and mark them as ongoing or completed once finished.
- The system will keep a record of all workers including their name, skill type, contact information, wage type (daily or hourly), and base rate.
- Project managers or site engineers will be able to assign workers to specific tasks for a fixed time period.
- Attendance of workers will be taken on a daily basis, including status (present, half-day, absent) and hours worked.
- Based on attendance and worker rate, the system will automatically calculate the total labor cost for that day.
- Vendors will be added to the system with their details like name, contact number, email, address, and rating.
- The project manager will be able to raise procurement requests to purchase materials from vendors by mentioning quantity, cost, and total amount.
- The system will maintain a list of all materials and items with their category, unit of measurement, estimated cost, and minimum stock quantity.
- Materials issued to a site or for a particular task will be recorded with issue date, item details, quantity, and the person who issued it.
- Material usage for each project or task will be tracked to know total consumption and remaining stock.

- The system will store all financial transactions for every project like material cost, labor cost, and vendor payments.
- Finance records will automatically get updated through attendance, procurement, and material issue entries to keep total cost accurate.
- The project manager will be able to view reports on project expenses, total cost, and budget utilization.
- The system will generate project-wise and vendor-wise reports which can be exported in formats like PDF or Excel.
- Managers can compare the actual expenditure of a project with the planned budget to analyze performance.
- Every record created or updated in the system will include user details and time stamps for transparency.
- The system will ensure that all data such as workers, projects, and vendors are properly linked through their IDs to maintain data integrity.
- Only authorized users will have rights to edit or delete records to prevent data misuse or errors.
- The overall system will help in managing projects, tracking work progress, handling inventory, and controlling costs effectively.

CHAPTER 2

2. SYSTEM DESIGN

2.1 ENTITY SETS

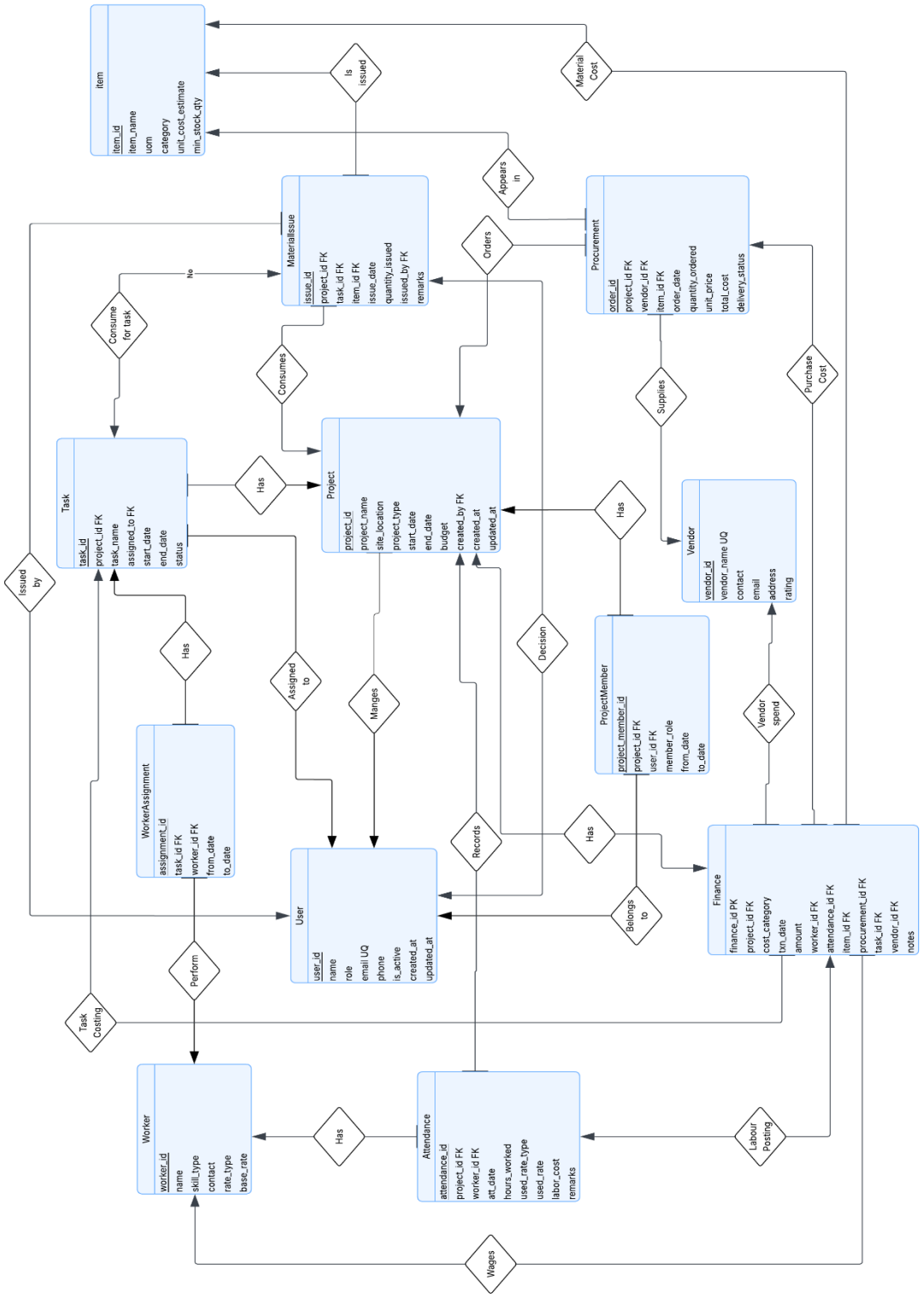
No.	Entity Set	Attributes
1	User	user_id (PK), name, role, email (UQ), phone, is_active, created_at, updated_at
2	Project	project_id (PK), project_name, site_location, project_type, start_date, end_date, budget, created_by (FK→User.user_id), created_at, updated_at
3	ProjectMember	project_member_id (PK), project_id (FK→Project.project_id), user_id (FK→User.user_id), member_role, from_date, to_date
4	Task	task_id (PK), project_id (FK→Project.project_id), task_name, assigned_to (FK→User.user_id), start_date, end_date, status
5	Worker	worker_id (PK), name, skill_type, contact, rate_type, base_rate
6	WorkerAssignment	assignment_id (PK), task_id (FK→Task.task_id), worker_id (FK→Worker.worker_id), from_date, to_date
7	Item	item_id (PK), item_name, uom, category, unit_cost_estimate, min_stock_qty
8	Vendor	vendor_id (PK), vendor_name (UQ), contact, email, address, rating
9	Procurement	order_id (PK), project_id (FK→Project.project_id), vendor_id (FK→Vendor.vendor_id), item_id (FK→Item.item_id), order_date, quantity_ordered, unit_price, total_cost, delivery_status
10	MaterialIssue	issue_id (PK), project_id (FK→Project.project_id), task_id (FK→Task.task_id), item_id (FK→Item.item_id), issue_date, quantity_issued, issued_by (FK→User.user_id)
11	Attendance	attendance_id (PK), project_id (FK→Project.project_id), worker_id (FK→Worker.worker_id), att_date, hours_worked, used_rate_type, used_rate, labor_cost
12	Finance	finance_id (PK), project_id (FK→Project.project_id), cost_category, txn_date, amount, worker_id (FK→Worker.worker_id), attendance_id (FK→Attendance.attendance_id), item_id (FK→Item.item_id), procurement_id (FK→Procurement.order_id), task_id (FK→Task.task_id), vendor_id (FK→Vendor.vendor_id)

2.2 RELATIONAL SETS

Associations	Relationship Name	Mapping Cardinality
User & Project	manages	One to Many (User → Project)
Project & ProjectMember	Has	One to Many (Project → ProjectMember)
User & ProjectMember	Belongs to	One to Many (User → ProjectMember)
Project & Task	Has	One to Many (Project → Task)
User & Task	assigned_to	One to Many (User → Task)
Task & Worker (via WorkerAssignment)	works_on	Many to Many (Task ↔ Worker)
Task & WorkerAssignment	Has	One to Many (Task → WorkerAssignment)
Worker & WorkerAssignment	Perform	One to Many (Worker → WorkerAssignment)
Project & Procurement	orders	One to Many (Project → Procurement)
Vendor & Procurement	supplies	One to Many (Vendor → Procurement)
Item & Procurement	appears_in	One to Many (Item → Procurement)
Project & MaterialIssue	consumes	One to Many (Project → MaterialIssue)
Task & MaterialIssue	consumes_for_task	One to Many (Task → MaterialIssue) (optional on MaterialIssue)
Item & MaterialIssue	is_issued	One to Many (Item → MaterialIssue)
User & MaterialIssue	issued_by	One to Many (User → MaterialIssue)
Project & Attendance	records	One to Many (Project → Attendance)
Worker & Attendance	has	One to Many (Worker → Attendance)
Project & Finance	has	One to Many (Project →

		Finance)
Attendance & Finance	labour_posting	One to One / One to Many (Attendance → Finance) (nullable FK in Finance)
Procurement & Finance	purchase_cost	One to Many (Procurement → Finance)
Task & Finance	task_cost	One to Many (Task → Finance)
Vendor & Finance	vendor_spend	One to Many (Vendor → Finance)
Item & Finance	material_cost	One to Many (Item → Finance)

2.3 ENTITY RELATIONSHIP DIAGRAM (E R-Diagram)



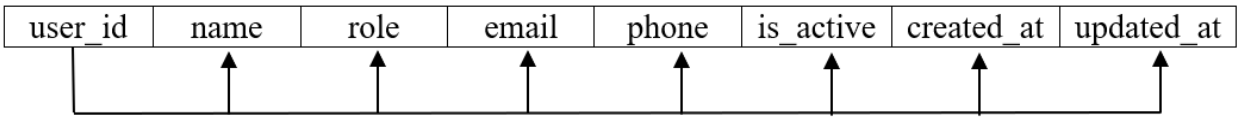
2.4 RELATIONAL TABLES

No.	Relational Table	Attributes (for the Database)
1	User	UserID, Name, Role, Email, Phone, IsActive, CreatedAt, UpdatedAt
2	Project	ProjectID, ProjectName, SiteLocation, ProjectType, StartDate, EndDate, Budget, CreatedBy (FK → User.UserID), CreatedAt, UpdatedAt
3	ProjectMember	ProjectMemberID, ProjectID (FK → Project.ProjectID), UserID (FK → User.UserID), MemberRole, FromDate, ToDate
4	Task	TaskID, ProjectID (FK → Project.ProjectID), TaskName, AssignedTo (FK → User.UserID), StartDate, EndDate, Status
5	Worker	WorkerID, Name, SkillType, Contact, RateType, BaseRate
6	WorkerAssignment	AssignmentID, TaskID (FK → Task.TaskID), WorkerID (FK → Worker.WorkerID), FromDate, ToDate
7	WorkerProject	WorkerID (FK → Worker.WorkerID), ProjectID (FK → Project.ProjectID), Primary Key (WorkerID, ProjectID)
8	Item	ItemID, ItemName, UOM, Category, UnitCostEstimate, MinStockQty
9	Vendor	VendorID, VendorName, Contact, Email, Address, Rating
10	Procurement	OrderID, ProjectID (FK → Project.ProjectID), VendorID (FK → Vendor.VendorID), ItemID (FK → Item.ItemID), OrderDate, QuantityOrdered, UnitPrice
11	ProcurementCost	OrderID (FK → Procurement.OrderID), TotalCost
12	MaterialIssue	IssueID, ProjectID (FK → Project.ProjectID), TaskID (FK → Task.TaskID), ItemID (FK → Item.ItemID), IssueDate, QuantityIssued, IssuedBy (FK → User.UserID), Remarks
13	Attendance	AttendanceID, ProjectID (FK → Project.ProjectID), WorkerID (FK → Worker.WorkerID), AttDate, HoursWorked, UsedRateType, UsedRate, LaborCost, Remarks

14	Finance	FinanceID, ProjectID (FK → Project.ProjectID), CostCategory, TxnDate, Amount, WorkerID (FK → Worker.WorkerID), AttendanceID (FK → Attendance.AttendanceID), ItemID (FK → Item.ItemID), ProcurementID (FK → Procurement.OrderID), TaskID (FK → Task.TaskID), VendorID (FK → Vendor.VendorID), Notes

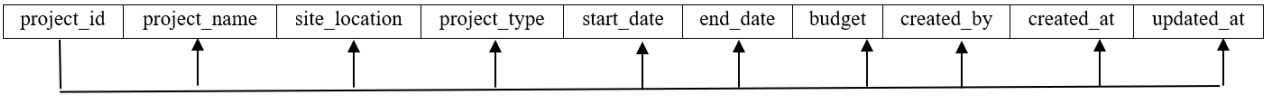
2.5 NORMALIZATION

1. User



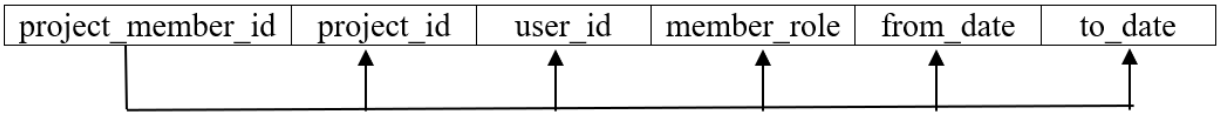
- **Attributes:** user_id, name, role, email, phone, is_active, created_at, updated_at.
- **Candidate Key:** user_id.
- **Functional Dependency:** {user_id}+ → {name, role, email, phone, is_active, created_at, updated_at}.
- **Normalization:** 1NF, 2NF, 3NF, BCNF.

2. Project



- **Attributes:** project_id, project_name, site_location, project_type, start_date, end_date, budget, created_by, created_at, updated_at.
- **Candidate Key:** project_id.
- **Functional Dependency:** {project_id}+ → {project_name, site_location, project_type, start_date, end_date, budget, created_by, created_at, updated_at}.
- **Normalization:** 1NF, 2NF, 3NF, BCNF.

3. ProjectMember



- **Attributes:** project_member_id, project_id, user_id, member_role, from_date, to_date.
- **Candidate Key:** project_member_id.
- **Functional Dependency:** {project_member_id}+ → {project_id, user_id, member_role, from_date, to_date
- **Normalization:** 1NF, 2NF, 3NF, BCNF.

4. Task

task_id	project_id	task_name	assigned_to	start_date	end_date	status

- **Attributes:** task_id, project_id, task_name, assigned_to, start_date, end_date, status.
- **Candidate Key:** task_id.
- **Functional Dependency:** {task_id}+ → {project_id, task_name, assigned_to, start_date, end_date, status}.
- **Normalization:** 1NF, 2NF, 3NF, BCNF.

5. Worker (Before Normalization)

- **Attributes:** worker_id, name, skill_type, contact, rate_type, base_rate, project_id.
- **Candidate Key:** worker_id.
- **Functional Dependency:** {worker_id}+ → {name, skill_type, contact, rate_type, base_rate, project_id}.
- **Normalization:** Not in 3NF/BCNF (project_id varies per project → redundancy). **Decompose** to Worker and WorkerProject.

5.a Worker

worker_id	name	skill_type	contact	rate_type	base_rate

- **Attributes:** worker_id, name, skill_type, contact, rate_type, base_rate.
- **Candidate Key:** worker_id.
- **Functional Dependency:** {worker_id}+ → {name, skill_type, contact, rate_type, base_rate}.
- **Normalization:** 1NF, 2NF, 3NF, BCNF.

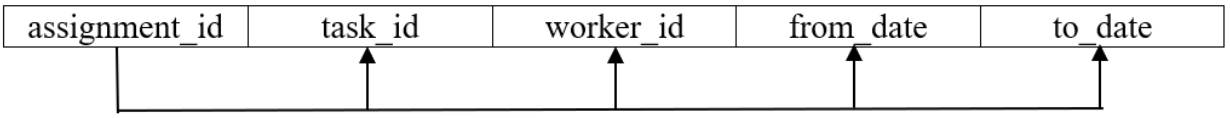
5.b WorkerProject

worker_id	project_id

- **Attributes:** worker_id, project_id.
- **Candidate Key:** {worker_id, project_id}.
- **Functional Dependency:** {worker_id, project_id}+ → {}.

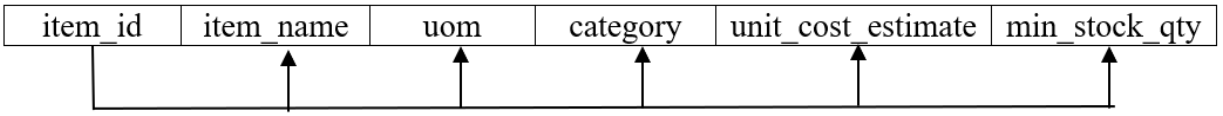
- **Normalization:** 1NF, 2NF, 3NF, BCNF.

6. WorkerAssignment



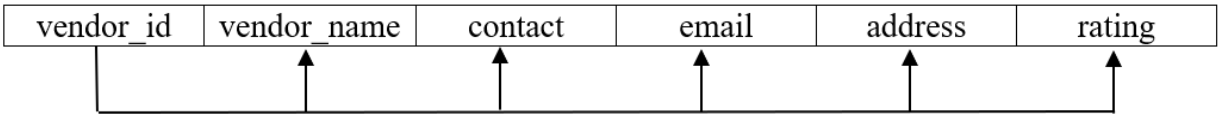
- **Attributes:** assignment_id, task_id, worker_id, from_date, to_date.
- **Candidate Key:** assignment_id.
- **Functional Dependency:** {assignment_id}+ → {task_id, worker_id, from_date, to_date}.
- **Normalization:** 1NF, 2NF, 3NF, BCNF.

7. Item



- **Attributes:** item_id, item_name, uom, category, unit_cost_estimate, min_stock_qty.
- **Candidate Key:** item_id.
- **Functional Dependency:** {item_id}+ → {item_name, uom, category, unit_cost_estimate, min_stock_qty}.
- **Normalization:** 1NF, 2NF, 3NF, BCNF.

8. Vendor

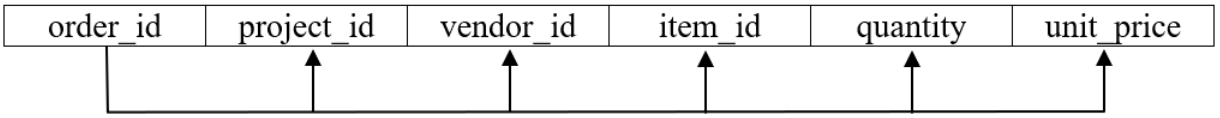


- **Attributes:** vendor_id, vendor_name, contact, email, address, rating.
- **Candidate Key:** vendor_id.
- **Functional Dependency:** {vendor_id}+ → {vendor_name, contact, email, address, rating}.
- **Normalization:** 1NF, 2NF, 3NF, BCNF.

9. Procurement (Before Normalization)

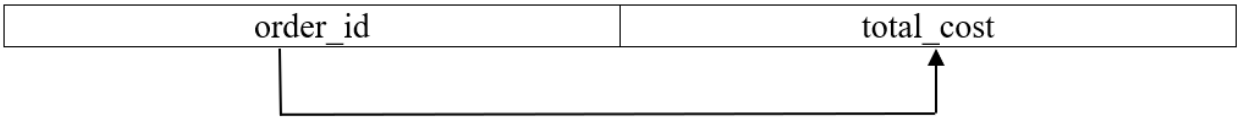
- **Attributes:** order_id, project_id, vendor_id, item_id, quantity, unit_price, total_cost.
- **Candidate Key:** order_id.
- **Functional Dependency:** {order_id}+ → {project_id, vendor_id, item_id, quantity, unit_price, total_cost}.
- **Normalization:** Not in 3NF/BCNF (total_cost is derived). **Decompose** to Procurement and ProcurementCost.

9.a Procurement



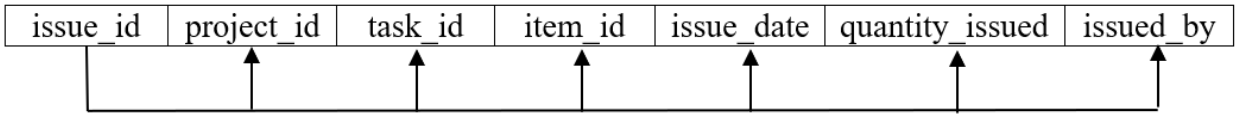
- **Attributes:** order_id, project_id, vendor_id, item_id, quantity, unit_price.
- **Candidate Key:** order_id.
- **Functional Dependency:** {order_id}+ → {project_id, vendor_id, item_id, quantity, unit_price}.
- **Normalization:** 1NF, 2NF, 3NF, BCNF.

9.b ProcurementCost



- **Attributes:** order_id, total_cost.
- **Candidate Key:** order_id.
- **Functional Dependency:** {order_id}+ → {total_cost}.
- **Normalization:** 1NF, 2NF, 3NF, BCNF.

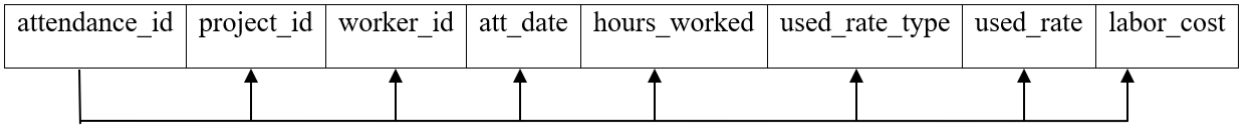
10. MaterialIssue



- **Attributes:** issue_id, project_id, task_id, item_id, issue_date, quantity_issued, issued_by.
- **Candidate Key:** issue_id.

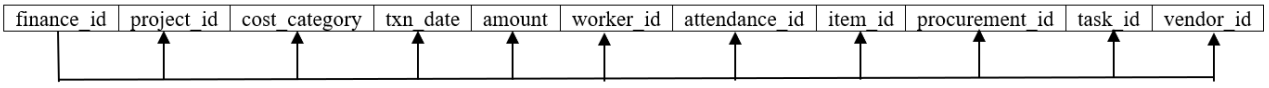
- **Functional Dependency:** {issue_id}+ → {project_id, task_id, item_id, issue_date, quantity_issued, issued_by}.
- **Normalization:** 1NF, 2NF, 3NF, BCNF.

11. Attendance



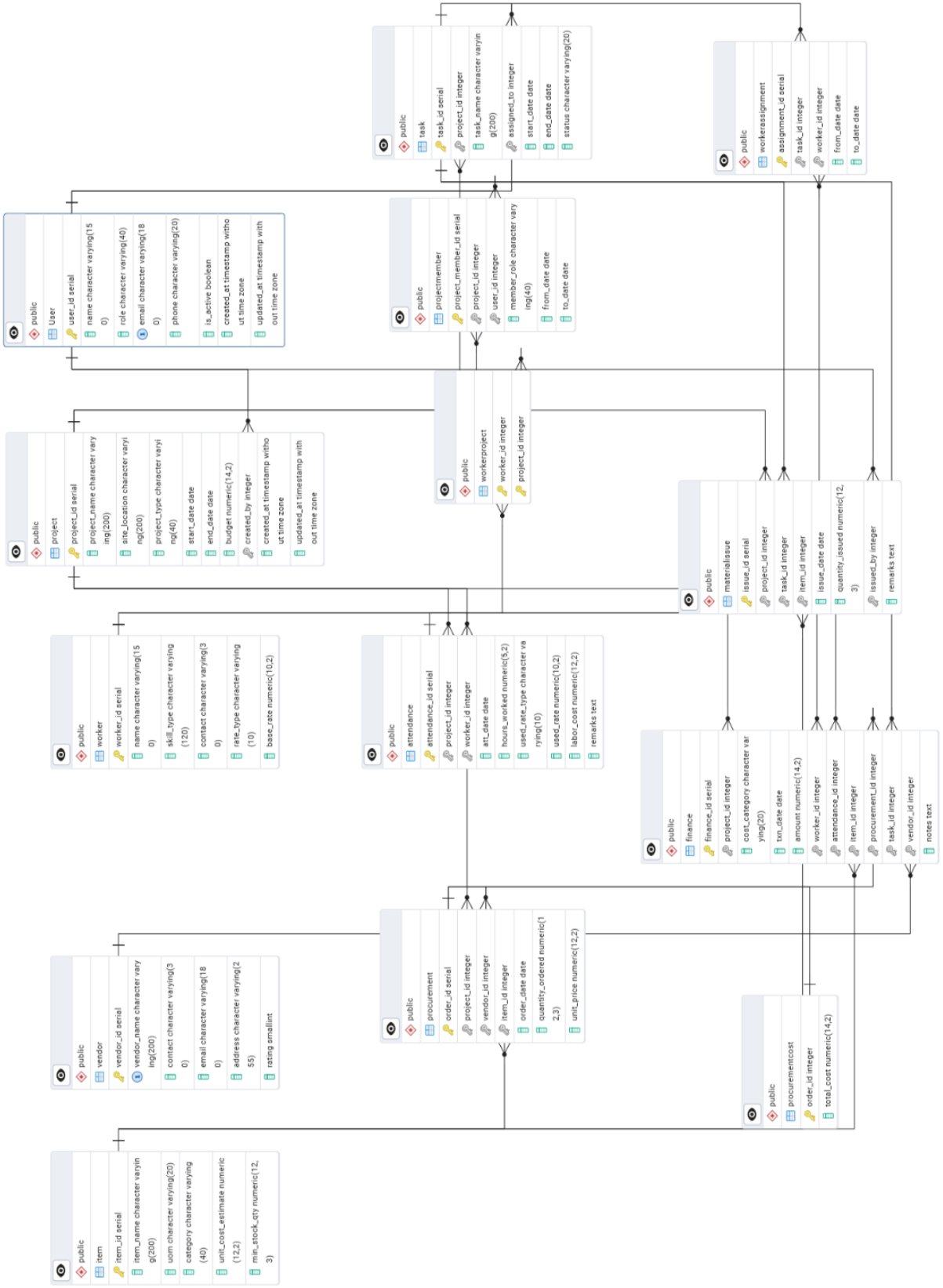
- **Attributes:** attendance_id, project_id, worker_id, att_date, hours_worked, used_rate_type, used_rate, labor_cost.
- **Candidate Key:** attendance_id.
- **Functional Dependency:** {attendance_id}+ → {project_id, worker_id, att_date, hours_worked, used_rate_type, used_rate, labor_cost}.
- **Normalization:** 1NF, 2NF, 3NF, BCNF.

12. Finance



- **Attributes:** finance_id, project_id, cost_category, txn_date, amount, worker_id, attendance_id, item_id, procurement_id, task_id, vendor_id, notes.
- **Candidate Key:** finance_id.
- **Functional Dependency:** {finance_id}+ → {project_id, cost_category, txn_date, amount, worker_id, attendance_id, item_id, procurement_id, task_id, vendor_id, notes}.
- **Normalization:** 1NF, 2NF, 3NF, BCNF.

2.6 RELATIONAL MODEL



CHAPTER 3

3. SCHEMA IN POSTGRESQL

3.1 CREATE TABLES & INSERT RECORDS

1. User Table

```sql
CREATE TABLE "User" (
  user_id       SERIAL PRIMARY KEY,
  name          VARCHAR(150) NOT NULL,
  role          VARCHAR(40)  NOT NULL,
  email         VARCHAR(180) NOT NULL UNIQUE,
  phone         VARCHAR(20),
  is_active     BOOLEAN NOT NULL DEFAULT TRUE,
  created_at    TIMESTAMP DEFAULT NOW(),
  updated_at    TIMESTAMP DEFAULT NOW()
);
```

2. Project Table

```sql
CREATE TABLE Project (
  project_id    SERIAL PRIMARY KEY,
  project_name  VARCHAR(200) NOT NULL,
  site_location VARCHAR(200),
  project_type  VARCHAR(40),
  start_date    DATE,
  end_date      DATE,
  budget        NUMERIC(14,2),
  created_by    INTEGER NOT NULL REFERENCES "User"(user_id),
  created_at    TIMESTAMP DEFAULT NOW(),
  updated_at    TIMESTAMP DEFAULT NOW()
);
```

3. ProjectMember Table

```sql
CREATE TABLE ProjectMember (
  project_member_id SERIAL PRIMARY KEY,
  project_id    INTEGER NOT NULL REFERENCES Project(project_id),
  user_id       INTEGER NOT NULL REFERENCES "User"(user_id),
  member_role   VARCHAR(40) NOT NULL,
  from_date     DATE,
  to_date       DATE
);
```

4. Task Table

```sql
CREATE TABLE Task (
  task_id       SERIAL PRIMARY KEY,
  project_id    INTEGER NOT NULL REFERENCES Project(project_id),
  task_name     VARCHAR(200) NOT NULL,
  assigned_to   INTEGER REFERENCES "User"(user_id),
  start_date    DATE,
  end_date      DATE,
  status        VARCHAR(20) NOT NULL DEFAULT 'open'
);
```

5. Worker Table

```sql
CREATE TABLE Worker (
  worker_id     SERIAL PRIMARY KEY,
  name          VARCHAR(150) NOT NULL,
  skill_type    VARCHAR(120),
  contact       VARCHAR(30),
  rate_type     VARCHAR(10) NOT NULL CHECK (rate_type IN ('daily','hourly')),
  base_rate     NUMERIC(10,2) NOT NULL
);
```

6. WorkerAssignment Table

```sql
CREATE TABLE WorkerAssignment (
  assignment_id SERIAL PRIMARY KEY,
  task_id   INTEGER NOT NULL REFERENCES Task(task_id),
  worker_id INTEGER NOT NULL REFERENCES Worker(worker_id),
  from_date DATE NOT NULL,
  to_date   DATE
);
```

7. WorkerProject Table

```sql
CREATE TABLE WorkerProject (
  worker_id  INTEGER NOT NULL REFERENCES Worker(worker_id),
  project_id INTEGER NOT NULL REFERENCES Project(project_id),
  PRIMARY KEY(worker_id, project_id)
);
```

8.Item Table

```sql
CREATE TABLE Item (
  item_id             SERIAL PRIMARY KEY,
  item_name           VARCHAR(200) NOT NULL,
  uom                 VARCHAR(20),
  category            VARCHAR(40),
  unit_cost_estimate  NUMERIC(12,2),
  min_stock_qty       NUMERIC(12,3) DEFAULT 0
);
```

9.Vendor Table

```sql
CREATE TABLE Vendor (
  vendor_id   SERIAL PRIMARY KEY,
  vendor_name VARCHAR(200) NOT NULL UNIQUE,
  contact     VARCHAR(30),
  email       VARCHAR(180),
  address     VARCHAR(255),
  rating      SMALLINT
);
```

10.Procurement Table

```sql
CREATE TABLE Procurement (
  order_id         SERIAL PRIMARY KEY,
  project_id       INTEGER NOT NULL REFERENCES Project(project_id),
  vendor_id        INTEGER NOT NULL REFERENCES Vendor(vendor_id),
  item_id          INTEGER NOT NULL REFERENCES Item(item_id),
  order_date       DATE NOT NULL,
  quantity_ordered NUMERIC(12,3) NOT NULL CHECK (quantity_ordered > 0),
  unit_price       NUMERIC(12,2) NOT NULL CHECK (unit_price >= 0)
);
```

11. ProcurementCost Table

```sql
CREATE TABLE ProcurementCost (
  order_id   INTEGER PRIMARY KEY REFERENCES Procurement(order_id) ON DELETE CASCADE,
  total_cost NUMERIC(14,2) NOT NULL CHECK (total_cost >= 0)
);
```

12. MaterialIssue Table

```sql
CREATE TABLE MaterialIssue (
  issue_id        SERIAL PRIMARY KEY,
  project_id      INTEGER NOT NULL REFERENCES Project(project_id),
  task_id         INTEGER REFERENCES Task(task_id),
  item_id         INTEGER NOT NULL REFERENCES Item(item_id),
  issue_date      DATE NOT NULL,
  quantity_issued NUMERIC(12,3) NOT NULL CHECK (quantity_issued > 0),
  issued_by       INTEGER NOT NULL REFERENCES "User"(user_id),
  remarks         TEXT
);
```

13. Attendance Table

```sql
CREATE TABLE Attendance (
  attendance_id  SERIAL PRIMARY KEY,
  project_id     INTEGER NOT NULL REFERENCES Project(project_id),
  worker_id      INTEGER NOT NULL REFERENCES Worker(worker_id),
  att_date       DATE NOT NULL,
  hours_worked   NUMERIC(5,2),
  used_rate_type VARCHAR(10) NOT NULL CHECK (used_rate_type IN ('daily','hourly')),
  used_rate      NUMERIC(10,2) NOT NULL,
  labor_cost     NUMERIC(12,2) NOT NULL,
  remarks        TEXT
);
```

14. Finance Table

```sql
CREATE TABLE Finance (
  finance_id     SERIAL PRIMARY KEY,
  project_id     INTEGER NOT NULL REFERENCES Project(project_id)
                  ON DELETE CASCADE ON UPDATE CASCADE,
  cost_category  VARCHAR(20) NOT NULL,
                  -- Expected values: 'labor', 'material', 'equipment', 'subcontract', 'overhead', 'other'
  txn_date       DATE NOT NULL,
  amount         NUMERIC(14,2) NOT NULL CHECK (amount >= 0),
  worker_id      INTEGER REFERENCES Worker(worker_id)
                  ON DELETE SET NULL ON UPDATE CASCADE,
  attendance_id  INTEGER REFERENCES Attendance(attendance_id)
                  ON DELETE SET NULL ON UPDATE CASCADE,
  item_id        INTEGER REFERENCES Item(item_id)
                  ON DELETE SET NULL ON UPDATE CASCADE,
  procurement_id INTEGER REFERENCES Procurement(order_id)
                  ON DELETE SET NULL ON UPDATE CASCADE,
  task_id        INTEGER REFERENCES Task(task_id)
                  ON DELETE SET NULL ON UPDATE CASCADE,
  vendor_id      INTEGER REFERENCES Vendor(vendor_id)
                  ON DELETE SET NULL ON UPDATE CASCADE,
  notes          TEXT,
  CONSTRAINT chk_cost_category CHECK (cost_category IN
      ('labor', 'material', 'equipment', 'subcontract', 'overhead', 'other'))
);
```

1. Insert User

```sql
INSERT INTO "User"(name, role, email, phone) VALUES
 ('Neel Patel','Admin','neel.patel@smguj.in','+91-97250-11111'),
 ('Krupa Shah','ProjectManager','krupa.shah@smguj.in','+91-98795-22222'),
 ('Viral Joshi','SiteEngineer','viral.joshi@smguj.in','+91-90330-33333'),
 ('Dhruvi Mehta','Storekeeper','dhruvi.mehta@smguj.in','+91-94280-44444');
```

- **Output**

	user_id [PK] integer	name character varying (150)	role character varying (40)	email character varying (180)	phone character varying (20)	is_active boolean	created_at timestamp without time zone	updated_at timestamp without time zone
1	1	Neel Patel	Admin	neel.patel@smguj.in	+91-97250-11111	true	2025-11-09 18:50:14.451751	2025-11-09 18:50:14.451751
2	2	Krupa Shah	ProjectManager	krupa.shah@smguj.in	+91-98795-22222	true	2025-11-09 18:50:14.451751	2025-11-09 18:50:14.451751
3	3	Viral Joshi	SiteEngineer	viral.joshi@smguj.in	+91-90330-33333	true	2025-11-09 18:50:14.451751	2025-11-09 18:50:14.451751
4	4	Dhruvi Mehta	Storekeeper	dhruvi.mehta@smguj.in	+91-94280-44444	true	2025-11-09 18:50:14.451751	2025-11-09 18:50:14.451751

2. Insert Project

```sql
INSERT INTO Project(project_name, site_location, project_type, start_date, budget, created_by) VALUES
 ('Gota Housing Block A','Gota, Ahmedabad','construction','2025-01-10',12000000,1),
 ('Metro Depot Shed','Sachin, Surat','mechanical','2025-02-01',6500000,2),
 ('Switchgear Panel Upgrade','Makarpura, Vadodara','electronics','2025-03-05',2800000,2);
```

- **Output**

	project_id [PK] integer	project_name character varying (200)	site_location character varying (200)	project_type character varying (40)	start_date date	end_date date	budget numeric (14,2)	created_by integer	created_at timestamp without time zone	updated_at timestamp without time zone
1	1	Gota Housing Block A	Gota, Ahmedabad	construction	2025-01-10	[null]	12000000.00	1	2025-11-09 18:52:23.375367	2025-11-09 18:52:23.375367
2	2	Metro Depot Shed	Sachin, Surat	mechanical	2025-02-01	[null]	6500000.00	2	2025-11-09 18:52:23.375367	2025-11-09 18:52:23.375367
3	3	Switchgear Panel Upgra...	Makarpura, Vadodara	electronics	2025-03-05	[null]	2800000.00	2	2025-11-09 18:52:23.375367	2025-11-09 18:52:23.375367

3. Insert ProjectMember

```
INSERT INTO ProjectMember(project_id, user_id, member_role, from_date) VALUES
 (1,2,'ProjectManager','2025-01-10'),
 (1,3,'SiteEngineer','2025-01-12'),
 (1,4,'Storekeeper','2025-01-12'),
 (2,2,'ProjectManager','2025-02-01'),
 (3,2,'ProjectManager','2025-03-05');
```

- **Output**

	project_id [PK] integer	project_name character varying (200)	site_location character varying (200)	project_type character varying (40)	start_date date	end_date date	budget numeric (14,2)	created_by integer	created_at timestamp without time zone	updated_at timestamp without time zone
1	1	Gota Housing Block A	Gota, Ahmedabad	construction	2025-01-10	[null]	12000000.00	1	2025-11-09 18:52:23.375367	2025-11-09 18:52:23.375367
2	2	Metro Depot Shed	Sachin, Surat	mechanical	2025-02-01	[null]	6500000.00	2	2025-11-09 18:52:23.375367	2025-11-09 18:52:23.375367
3	3	Switchgear Panel Upgra...	Makarpura, Vadodara	electronics	2025-03-05	[null]	2800000.00	2	2025-11-09 18:52:23.375367	2025-11-09 18:52:23.375367

4. Insert Task

```
INSERT INTO Task(project_id, task_name, assigned_to, start_date, status) VALUES
 (1,'Excavation & PCC', 3, '2025-01-15','in_progress'),
 (1,'Brick Masonry GF', 3, '2025-01-25','open'),
 (2,'Fabricate Truss',  3, '2025-02-05','open'),
 (3,'Panel Wiring',     3, '2025-03-10','open');
```

- **Output**

	task_id [PK] integer	project_id integer	task_name character varying (200)	assigned_to integer	start_date date	end_date date	status character varying (20)
1	1	1	Excavation & PCC	3	2025-01-15	[null]	in_progress
2	2	1	Brick Masonry GF	3	2025-01-25	[null]	open
3	3	2	Fabricate Truss	3	2025-02-05	[null]	open
4	4	3	Panel Wiring	3	2025-03-10	[null]	open

5. Insert Worker

```
INSERT INTO Worker(name, skill_type, contact, rate_type, base_rate) VALUES
 ('Rakesh Patel','Mason','98980-11111','daily',900),
 ('Jignesh Chauhan','Helper','98980-22222','daily',600),
 ('Mehul Shah','Welder','98980-33333','hourly',120),
 ('Amit Solanki','Electrician','98980-44444','hourly',150);
```

➢ **Output**

	worker_id [PK] integer	name character varying (150)	skill_type character varying (120)	contact character varying (30)	rate_type character varying (10)	base_rate numeric (10,2)
1	1	Rakesh Patel	Mason	98980-11111	daily	900.00
2	2	Jignesh Chauhan	Helper	98980-22222	daily	600.00
3	3	Mehul Shah	Welder	98980-33333	hourly	120.00
4	4	Amit Solanki	Electrician	98980-44444	hourly	150.00

6. Insert WorkerAssignment

```
INSERT INTO WorkerAssignment(task_id, worker_id, from_date) VALUES
 (1,1,'2025-01-16'),
 (1,2,'2025-01-16'),
 (3,3,'2025-02-06'),
 (4,4,'2025-03-12');
```

➢ **Output**

	assignment_id [PK] integer	task_id integer	worker_id integer	from_date date	to_date date
1	1	1	1	2025-01-16	[null]
2	2	1	2	2025-01-16	[null]
3	3	3	3	2025-02-06	[null]
4	4	4	4	2025-03-12	[null]

7. Insert WorkerProject

```
INSERT INTO WorkerProject(worker_id, project_id) VALUES
 (1,1),(2,1),(3,2),(4,3);
```

➢ **Output**

	worker_id [PK] integer	project_id [PK] integer
1	1	1
2	2	1
3	3	2
4	4	3

8. Insert Item

```
INSERT INTO Item(item_name, uom, category, unit_cost_estimate, min_stock_qty) VALUES
 ('OPC Cement 53 Grade','BAG','material',380,100),
 ('First Class Bricks','NOS','material',6.5,5000),
 ('TMT Steel 12mm','KG','material',62,2000),
 ('4C x 10 sqmm Cable','MTR','material',120,500);
```

- **Output**

	item_id [PK] integer	item_name character varying (200)	uom character varying (20)	category character varying (40)	unit_cost_estimate numeric (12,2)	min_stock_qty numeric (12,3)
1	1	OPC Cement 53 Grade	BAG	material	380.00	100.000
2	2	First Class Bricks	NOS	material	6.50	5000.000
3	3	TMT Steel 12mm	KG	material	62.00	2000.000
4	4	4C x 10 sqmm Cable	MTR	material	120.00	500.000

9. Insert Vendor

```
INSERT INTO Vendor(vendor_name, contact, email, address, rating) VALUES
 ('Shree Shakti Traders','98250-11122','sales@shreeshakti.in','Naroda, Ahmedabad, Gujarat',5),
 ('Surat Steel Mart','98250-33344','info@suratsteelmart.in','Udhna, Surat, Gujarat',4),
 ('Vadodara Electricals','98250-55566','support@vadelec.in','Makarpura, Vadodara, Gujarat',5);
```

- **Output**

	vendor_id [PK] integer	vendor_name character varying (200)	contact character varying (30)	email character varying (180)	address character varying (255)	rating smallint
1	1	Shree Shakti Traders	98250-11122	sales@shreeshakti.in	Naroda, Ahmedabad, Gujarat	5
2	2	Surat Steel Mart	98250-33344	info@suratsteelmart.in	Udhna, Surat, Gujarat	4
3	3	Vadodara Electricals	98250-55566	support@vadelec.in	Makarpura, Vadodara, Guja...	5

10.Insert Procurement

```
INSERT INTO Procurement(project_id, vendor_id, item_id, order_date, quantity_ordered, unit_price) VALUES
 (1, 1, 1, '2025-01-18', 600, 375),
 (1, 1, 2, '2025-01-22', 8000, 6.2),
 (2, 2, 3, '2025-02-07', 2500, 61.5),
 (3, 3, 4, '2025-03-12', 700, 118);
```

- **Output**

	order_id [PK] integer	project_id integer	vendor_id integer	item_id integer	order_date date	quantity_ordered numeric (12,3)	unit_price numeric (12,2)
1	1	1	1	1	2025-01-18	600.000	375.00
2	2	1	1	2	2025-01-22	8000.000	6.20
3	3	2	2	3	2025-02-07	2500.000	61.50
4	4	3	3	4	2025-03-12	700.000	118.00

11.Insert ProcurementCost

```
INSERT INTO ProcurementCost(order_id, total_cost) VALUES
 (1, 600*375),
 (2, 8000*6.2),
 (3, 2500*61.5),
 (4, 700*118);
```

- **Output**

	order_id [PK] integer	total_cost numeric (14,2)
1	1	225000.00
2	2	49600.00
3	3	153750.00
4	4	82600.00

12. Insert MaterailIssue

```
INSERT INTO MaterialIssue(project_id, task_id, item_id, issue_date, quantity_issued, issued_by, remarks) VALUES
 (1, 1, 1, '2025-01-19', 120, 4, 'Cement for PCC'),
 (1, 2, 2, '2025-01-27', 2500, 4, 'Bricks for masonry'),
 (2, 3, 3, '2025-02-10', 900, 4, 'Steel for truss'),
 (3, NULL, 4, '2025-03-15', 200, 4, 'Cable issued to site store');
```

- **Output**

	issue_id [PK] integer	project_id integer	task_id integer	item_id integer	issue_date date	quantity_issued numeric (12,3)	issued_by integer	remarks text
1	1	1	1	1	2025-01-19	120.000	4	Cement for PCC
2	2	1	2	2	2025-01-27	2500.000	4	Bricks for masonry
3	3	2	3	3	2025-02-10	900.000	4	Steel for truss
4	4	3	[null]	4	2025-03-15	200.000	4	Cable issued to site st...

13.Insert Attendance

```
INSERT INTO Attendance(project_id, worker_id, att_date, hours_worked, used_rate_type, used_rate, labor_cost, remarks) VALUES
 (1, 1, '2025-01-19', NULL, 'daily', 900, 900, 'Present'),
 (1, 2, '2025-01-19', NULL, 'daily', 600, 600, 'Present'),
 (3, 4, '2025-03-16', 7.5, 'hourly', 150, 1125, 'Wiring'),
 (2, 3, '2025-02-10', 8.0, 'hourly', 120, 960, 'Shop welding');
```

- **Output**

	attendance_id [PK] integer	project_id integer	worker_id integer	att_date date	hours_worked numeric (5,2)	used_rate_type character varying (10)	used_rate numeric (10,2)	labor_cost numeric (12,2)	remarks text
1	1	1	1	2025-01-19	[null]	daily	900.00	900.00	Present
2	2	1	2	2025-01-19	[null]	daily	600.00	600.00	Present
3	3	3	4	2025-03-16	7.50	hourly	150.00	1125.00	Wiring
4	4	2	3	2025-02-10	8.00	hourly	120.00	960.00	Shop welding

14.Finance

```sql
INSERT INTO Finance
(project_id, cost_category, txn_date, amount, worker_id, attendance_id, item_id, procurement_id, task_id, vendor_id, notes)
VALUES
(1, 'material', '2025-01-20', (SELECT total_cost FROM ProcurementCost WHERE order_id = 1),
    NULL, NULL,
    (SELECT item_id FROM Procurement WHERE order_id = 1),
    1,
    (SELECT task_id FROM Task WHERE project_id = 1 LIMIT 1),
    (SELECT vendor_id FROM Procurement WHERE order_id = 1),
    'Cement purchase for PCC'),
(1, 'material', '2025-01-25', (SELECT total_cost FROM ProcurementCost WHERE order_id = 2),
    NULL, NULL,
    (SELECT item_id FROM Procurement WHERE order_id = 2),
    2,
    (SELECT task_id FROM Task WHERE project_id = 1 AND task_name ILIKE '%Brick%' LIMIT 1),
    (SELECT vendor_id FROM Procurement WHERE order_id = 2),
    'Brick procurement for masonry'),
(2, 'material', '2025-02-10', (SELECT total_cost FROM ProcurementCost WHERE order_id = 3),
    NULL, NULL,
    (SELECT item_id FROM Procurement WHERE order_id = 3),
    3,
    (SELECT task_id FROM Task WHERE project_id = 2 LIMIT 1),
    (SELECT vendor_id FROM Procurement WHERE order_id = 3),
    'Steel material purchase for truss fabrication'),

(3, 'labor', '2025-03-16',
    (SELECT labor_cost FROM Attendance WHERE attendance_id = 3),
    (SELECT worker_id FROM Attendance WHERE attendance_id = 3),
    3, NULL, NULL,
    (SELECT task_id FROM Task WHERE project_id = 3 LIMIT 1),
    NULL,
    'Electrician labor for wiring'),
```

➢ Output

	finance_id [PK] integer	project_id integer	cost_category character varying (20)	txn_date date	amount numeric (14,2)	worker_id integer	attendance_id integer	item_id integer	procurement_id integer	task_id integer	vendor_id integer	notes text
1	1	1	material	2025-01-20	225000.00	[null]	[null]	1	1	1	1	Cement purchase for PCC
2	2	1	material	2025-01-25	49600.00	[null]	[null]	2	2	2	1	Brick procurement for masonry
3	3	2	material	2025-02-10	153750.00	[null]	[null]	3	3	3	2	Steel material purchase for truss fabrication
4	4	3	labor	2025-03-16	1125.00	4	3	[null]	[null]	4	[null]	Electrician labor for wiring

3.2 QUERIES WITH OUTPUTS TO FETCH VITAL RECORDS FROM DATABASE

1. List all projects with their project manager names

```
SELECT
    p.project_name,
    u.name AS project_manager
FROM
    Project p
JOIN
    ProjectMember pm ON p.project_id = pm.project_id
JOIN
    "User" u ON pm.user_id = u.user_id
WHERE
    pm.member_role LIKE 'ProjectManager';
```

- **Output**

	project_name character varying (200)	project_manager character varying (150)
1	Switchgear Panel Upgra...	Krupa Shah
2	Metro Depot Shed	Krupa Shah
3	Gota Housing Block A	Krupa Shah

2. Show total number of workers assigned to each project

```
SELECT
    p.project_name,
    COUNT(wp.worker_id) AS total_workers
FROM
    Project p
JOIN
    WorkerProject wp ON p.project_id = wp.project_id
GROUP BY
    p.project_name
ORDER BY
    total_workers DESC;
```

➢ **Output**

	project_name character varying (200)	total_workers bigint
1	Gota Housing Block A	2
2	Switchgear Panel Upgra...	1
3	Metro Depot Shed	1

3. List all procurement orders with vendor and item details

```
SELECT
    pr.order_id,
    p.project_name,
    v.vendor_name,
    i.item_name,
    pr.quantity_ordered,
    pr.unit_price,
    (pr.quantity_ordered * pr.unit_price) AS total_amount
FROM
    Procurement pr
JOIN
    Project p ON pr.project_id = p.project_id
JOIN
    Vendor v ON pr.vendor_id = v.vendor_id
JOIN
    Item i ON pr.item_id = i.item_id
ORDER BY
    pr.order_date;
```

➢ **Output**

	order_id integer	project_name character varying (200)	vendor_name character varying (200)	item_name character varying (200)	quantity_ordered numeric (12,3)	unit_price numeric (12,2)	total_amount numeric
1	1	Gota Housing Block A	Shree Shakti Traders	OPC Cement 53 Grade	600.000	375.00	225000.00000
2	2	Gota Housing Block A	Shree Shakti Traders	First Class Bricks	8000.000	6.20	49600.00000
3	3	Metro Depot Shed	Surat Steel Mart	TMT Steel 12mm	2500.000	61.50	153750.00000
4	4	Switchgear Panel Upgra...	Vadodara Electricals	4C x 10 sqmm Cable	700.000	118.00	82600.00000

4. Find total labor cost per project

```
SELECT
    p.project_name,
    SUM(a.labor_cost) AS total_labor_cost
FROM
    Attendance a
JOIN
    Project p ON a.project_id = p.project_id
GROUP BY
    p.project_name;
```

- **Output**

	project_name character varying (200)	total_labor_cost numeric
1	Gota Housing Block A	1500.00
2	Switchgear Panel Upgra...	1125.00
3	Metro Depot Shed	960.00

5. Retrieve projects having budget greater than average budget

```
SELECT
    project_name,
    budget
FROM
    Project
WHERE
    budget > (SELECT AVG(budget) FROM Project);
```

- **Output**

	project_name character varying (200)	budget numeric (14,2)
1	Gota Housing Block A	12000000.00

6. List tasks and assigned users where status is still open

```sql
SELECT
    t.task_name,
    t.status,
    u.name AS assigned_user,
    p.project_name
FROM
    Task t
JOIN
    "User" u ON t.assigned_to = u.user_id
JOIN
    Project p ON t.project_id = p.project_id
WHERE
    t.status LIKE 'open';
```

- **Output**

	task_name character varying (200)	status character varying (20)	assigned_user character varying (150)	project_name character varying (200)
1	Brick Masonry GF	open	Viral Joshi	Gota Housing Block A
2	Fabricate Truss	open	Viral Joshi	Metro Depot Shed
3	Panel Wiring	open	Viral Joshi	Switchgear Panel Upgra...

7. Get total procurement cost per vendor

```sql
SELECT
    v.vendor_name,
    SUM(pc.total_cost) AS total_procurement_value
FROM
    ProcurementCost pc
JOIN
    Procurement pr ON pc.order_id = pr.order_id
JOIN
    Vendor v ON pr.vendor_id = v.vendor_id
GROUP BY
    v.vendor_name
ORDER BY
    total_procurement_value DESC;
```

➢ **Output**

	vendor_name character varying (200)	total_procurement_value numeric
1	Shree Shakti Traders	274600.00
2	Surat Steel Mart	153750.00
3	Vadodara Electricals	82600.00

8. Show all finance transactions tagged as 'material' along with vendor and project

```
SELECT
    f.finance_id,
    p.project_name,
    v.vendor_name,
    f.amount,
    f.txn_date,
    f.notes
FROM
    Finance f
LEFT JOIN
    Project p ON f.project_id = p.project_id
LEFT JOIN
    Vendor v ON f.vendor_id = v.vendor_id
WHERE
    f.cost_category = 'material'
ORDER BY
    f.txn_date;
```

➢ **Output**

	finance_id integer	project_name character varying (200)	vendor_name character varying (200)	amount numeric (14,2)	txn_date date	notes text
1	1	Gota Housing Block A	Shree Shakti Traders	225000.00	2025-01-...	Cement purchase for PCC
2	2	Gota Housing Block A	Shree Shakti Traders	49600.00	2025-01-...	Brick procurement for masonry
3	3	Metro Depot Shed	Surat Steel Mart	153750.00	2025-02-...	Steel material purchase for truss fabricat...

9. List projects having at least one procurement order above the average order cost

```sql
SELECT
    p.project_name,
    p.site_location,
    p.budget
FROM
    Project p
WHERE
    p.project_id IN (
        SELECT
            project_id
        FROM
            Procurement pr
        JOIN
            ProcurementCost pc ON pr.order_id = pc.order_id
        GROUP BY
            project_id
        HAVING
            MAX(pc.total_cost) > (SELECT AVG(total_cost) FROM ProcurementCost)
    )
ORDER BY
    p.project_name;
```

- **Output**

	project_name character varying (200)	site_location character varying (200)	budget numeric (14,2)
1	Gota Housing Block A	Gota, Ahmedabad	12000000.00
2	Metro Depot Shed	Sachin, Surat	6500000.00

10. Find all workers who worked on projects in the city "Ahmedabad"

```sql
SELECT
    w.name AS worker_name,
    w.skill_type
FROM
    Worker w
WHERE
    EXISTS (
        SELECT 1
        FROM WorkerProject wp
        JOIN Project p ON wp.project_id = p.project_id
        WHERE
            wp.worker_id = w.worker_id
            AND p.site_location LIKE '%Ahmedabad%'
    );
```

- **Output**

	worker_name character varying (150)	skill_type character varying (120)
1	Rakesh Patel	Mason
2	Jignesh Chauhan	Helper

11.Insert Value in Worker

```
CREATE OR REPLACE FUNCTION insert_worker(
    p_name VARCHAR,
    p_skill_type VARCHAR,
    p_contact VARCHAR,
    p_rate_type VARCHAR,
    p_base_rate NUMERIC
)
RETURNS INTEGER
LANGUAGE plpgsql
AS $$
DECLARE
    v_worker_id INTEGER;
    v_rate_type TEXT;
BEGIN
    -- Basic validation
    IF p_name IS NULL OR trim(p_name) = '' THEN
        RAISE EXCEPTION 'Worker name cannot be empty';
    END IF;

    IF p_base_rate IS NULL OR p_base_rate <= 0 THEN
        RAISE EXCEPTION 'base_rate must be > 0';
    END IF;

    -- Normalize and validate rate_type
    v_rate_type := lower(coalesce(p_rate_type, ''));
    IF v_rate_type NOT IN ('daily','hourly') THEN
        RAISE EXCEPTION 'rate_type must be either "daily" or "hourly"';
    END IF;

    -- Insert
    INSERT INTO Worker(name, skill_type, contact, rate_type, base_rate)
    VALUES (p_name, p_skill_type, p_contact, v_rate_type, p_base_rate)
    RETURNING worker_id INTO v_worker_id;

    RAISE NOTICE 'Worker "%" inserted with worker_id=%', p_name, v_worker_id;
    RETURN v_worker_id;
END;
$$;
```

- **Output**

```
105
106  SELECT insert_worker('Bhavesh Desai', 'Electrician', '98980-66666', 'hourly', 140);
```

Data Output | Messages | Notifications

```
NOTICE:  Worker "Bhavesh Desai" inserted with worker_id=5

Successfully run. Total query runtime: 53 msec.
1 rows affected.
```

12. Total Number of Tasks in a given project

```
CREATE OR REPLACE FUNCTION count_tasks_by_project(p_project_id INT)
RETURNS INT
LANGUAGE plpgsql
AS $$
DECLARE
    total_tasks INT;
BEGIN
    SELECT COUNT(*) INTO total_tasks
    FROM Task
    WHERE project_id = p_project_id;

    RAISE NOTICE 'Total tasks for project % = %', p_project_id, total_tasks;

    RETURN total_tasks;
END;
$$;
```

- **Output**

```
126    SELECT count_tasks_by_project(1);
```

Data Output Messages Notifications

	count_tasks_by_project integer
1	2

13. Assign a worker to a task

```
CREATE OR REPLACE PROCEDURE assign_worker_to_task(
    p_task_id   INT,
    p_worker_id INT,
    p_from      DATE,
    p_to        DATE DEFAULT NULL
)
LANGUAGE plpgsql
AS $$
BEGIN

  IF NOT EXISTS (SELECT 1 FROM Task   WHERE task_id = p_task_id)   THEN
    RAISE EXCEPTION 'Task % not found', p_task_id;
  END IF;
  IF NOT EXISTS (SELECT 1 FROM Worker WHERE worker_id = p_worker_id) THEN
    RAISE EXCEPTION 'Worker % not found', p_worker_id;
  END IF;


  INSERT INTO WorkerAssignment(task_id, worker_id, from_date, to_date)
  VALUES (p_task_id, p_worker_id, p_from, p_to);


  INSERT INTO WorkerProject(worker_id, project_id)
  SELECT p_worker_id, t.project_id
  FROM Task t
  WHERE t.task_id = p_task_id
  ON CONFLICT (worker_id, project_id) DO NOTHING;

  RAISE NOTICE 'Worker % assigned to task % from % to %', p_worker_id, p_task_id, p_from, p_to;
END;
$$;
```

- **Output**

```
159
160   CALL assign_worker_to_task(1, 1, DATE '2025-01-20', NULL);
```

Data Output | Messages | Notifications

```
NOTICE:  Worker 1 assigned to task 1 from 2025-01-20 to <NULL>
CALL

Query returned successfully in 39 msec.
```

14. Post a procurement to Finance

```
CREATE OR REPLACE PROCEDURE post_procurement_to_finance(
    p_order_id INT,
    p_txn_date DATE DEFAULT CURRENT_DATE
)
LANGUAGE plpgsql
AS $$
DECLARE
  v_project  INT;
  v_item     INT;
  v_vendor   INT;
  v_amount   NUMERIC(14,2);
BEGIN

  SELECT pr.project_id, pr.item_id, pr.vendor_id, pc.total_cost
  INTO   v_project,   v_item,    v_vendor,   v_amount
  FROM Procurement pr
  JOIN ProcurementCost pc ON pc.order_id = pr.order_id
  WHERE pr.order_id = p_order_id;

  IF v_project IS NULL OR v_amount IS NULL THEN
    RAISE EXCEPTION 'Procurement % not found or missing cost', p_order_id;
  END IF;


  INSERT INTO Finance(project_id, cost_category, txn_date, amount,
                      item_id, procurement_id, vendor_id, notes)
  VALUES (v_project, 'material', p_txn_date, v_amount,
          v_item, p_order_id, v_vendor, 'Auto-posted from procurement');

  RAISE NOTICE 'Finance posted for Procurement % with amount %', p_order_id, v_amount;
END;
$$;
```

- **Output**

```
195    CALL post_procurement_to_finance(1, DATE '2025-01-20');
```

Data Output Messages Notifications

```
NOTICE:  Finance posted for Procurement 1 with amount 225000.00
CALL

Query returned successfully in 50 msec.
```

15. Cursor to list all projects and their budgets

```
CREATE OR REPLACE PROCEDURE show_all_projects()
LANGUAGE plpgsql
AS $$
DECLARE
    rec RECORD;
    proj_cur CURSOR FOR
        SELECT project_name, budget FROM Project ORDER BY project_id;
BEGIN
    RAISE NOTICE '--- Project List ---';
    OPEN proj_cur;

    LOOP
        FETCH proj_cur INTO rec;
        EXIT WHEN NOT FOUND;
        RAISE NOTICE 'Project: %, Budget: ₹%', rec.project_name, rec.budget;
    END LOOP;

    CLOSE proj_cur;
    RAISE NOTICE '--- End of List ---';
END;
$$;
```

- **Output**

```
219    CALL show_all_projects();
```

Data Output | Messages | Notifications

```
NOTICE:  --- Project List ---
NOTICE:  Project: Gota Housing Block A, Budget: ₹12000000.00
NOTICE:  Project: Metro Depot Shed, Budget: ₹6500000.00
NOTICE:  Project: Switchgear Panel Upgrade, Budget: ₹2800000.00
NOTICE:  Project: Smart Housing Complex, Budget: ₹12500000.00
NOTICE:  --- End of List ---
CALL

Query returned successfully in 43 msec.
```

16. Cursor to show all workers assigned to a specific project

```
CREATE OR REPLACE PROCEDURE show_workers_by_project(p_project_id INT)
LANGUAGE plpgsql
AS $$
DECLARE
    rec RECORD;
    work_cur CURSOR FOR
        SELECT DISTINCT w.name, w.skill_type, w.rate_type
        FROM Worker w
        JOIN WorkerAssignment wa ON w.worker_id = wa.worker_id
        JOIN Task t ON wa.task_id = t.task_id
        WHERE t.project_id = p_project_id;
BEGIN
    RAISE NOTICE '--- Workers for Project ID % ---', p_project_id;
    OPEN work_cur;

    LOOP
        FETCH work_cur INTO rec;
        EXIT WHEN NOT FOUND;
        RAISE NOTICE 'Worker: %, Skill: %, Rate Type: %', rec.name, rec.skill_type, rec.rate_type;
    END LOOP;

    CLOSE work_cur;
    RAISE NOTICE '--- End of Worker List ---';
END;
$$;
```

- **Output**

```
CALL show_workers_by_project(1);
```

Data Output | Messages | Notifications

```
NOTICE:  --- Workers for Project ID 1 ---
NOTICE:  Worker: Jignesh Chauhan, Skill: Helper, Rate Type: daily
NOTICE:  Worker: Rakesh Patel, Skill: Mason, Rate Type: daily
NOTICE:  --- End of Worker List ---
CALL

Query returned successfully in 63 msec.
```

17. Trigger: Auto-update updated_at column when a Project record is modified

```
CREATE OR REPLACE FUNCTION update_project_timestamp()
RETURNS TRIGGER
LANGUAGE plpgsql
AS $$
BEGIN
    NEW.updated_at := NOW();
    RETURN NEW;
END;
$$;

CREATE TRIGGER trg_update_project_timestamp
BEFORE UPDATE
ON Project
FOR EACH ROW
EXECUTE FUNCTION update_project_timestamp();
```

- **Output**

```
UPDATE Project
SET budget = budget + 500000
WHERE project_id = 1;

SELECT project_id, project_name, budget, updated_at FROM Project;
```

Data Output Messages Notifications

	project_id [PK] integer	project_name character varying (200)	budget numeric (14,2)	updated_at timestamp without time zone
1	2	Metro Depot Shed	6500000.00	2025-11-09 18:52:23.375367
2	3	Switchgear Panel Upgra...	2800000.00	2025-11-09 18:52:23.375367
3	4	Smart Housing Complex	12500000.00	2025-11-10 21:53:07.408773
4	1	Gota Housing Block A	12500000.00	2025-11-10 22:40:37.315529

18. Trigger: Auto-calculate total cost in ProcurementCost after new Procurement

```
CREATE OR REPLACE FUNCTION insert_procurement_cost()
RETURNS TRIGGER
LANGUAGE plpgsql
AS $$
DECLARE
    v_total NUMERIC(14,2);
BEGIN
    v_total := NEW.quantity_ordered * NEW.unit_price;
    INSERT INTO ProcurementCost(order_id, total_cost)
    VALUES (NEW.order_id, v_total);

    RAISE NOTICE 'ProcurementCost inserted for order_id % with total = %', NEW.order_id, v_total;
    RETURN NEW;
END;
$$;
CREATE TRIGGER trg_insert_procurementcost
AFTER INSERT
ON Procurement
FOR EACH ROW
EXECUTE FUNCTION insert_procurement_cost();
```

- **Output**

```
INSERT INTO Procurement(project_id, vendor_id, item_id, order_date, quantity_ordered, unit_price)
VALUES (1, 1, 1, '2025-04-01', 500, 380);

SELECT * FROM ProcurementCost ORDER BY order_id DESC;
```

Data Output | Messages | Notifications

```
NOTICE:  ProcurementCost inserted for order_id 5 with total = 190000.00

Successfully run. Total query runtime: 62 msec.
5 rows affected.
```

	order_id [PK] integer	total_cost numeric (14,2)
1	5	190000.00
2	4	82600.00
3	3	153750.00
4	2	49600.00
5	1	225000.00